



An Improved Parallel Particle Swarm Optimization

Vasileios Charilogis¹ · Ioannis G. Tsoulos¹ · Alexandros Tzallas¹

Received: 12 April 2023 / Accepted: 5 August 2023
© The Author(s) 2023

Abstract

In the area of global optimization, a variety of techniques have been developed to find the global minimum. These techniques, in most cases, require a significant amount of computational resources and time to complete and therefore there is a need to develop parallel techniques. In addition, the wide spread of parallel architectures in recent years greatly facilitates the implementation of such techniques. Among the most widely used global optimization techniques is the particle swarm optimization technique. In this work, a series of modifications are proposed in the direction of efficient parallelization for particle swarm optimization. These modifications include an innovative velocity calculation mechanism that has also been successfully used in the serial version of the method, mechanisms for propagating the best particles between parallel computing units, but also a process termination mechanism, which has been properly configured for efficient execution in parallel computing environments. The proposed technique was applied to a multitude of computational problems from the relevant literature and the results were more than promising, since it was found that increasing the computational threads can significantly reduce the required number of function calls to find the global minimum. The proposed technique is at rate of 50–70% of the required number of function calls compared to other optimization techniques. This reduction is visible even if one to two parallel processing units are used. In addition, with the increase in parallel processing units, a drastic reduction in the number of calls is observed and therefore a reduction in the required computing time, which can reach up to 70%.

Keywords Optimization · Parallel methods · Evolutionary techniques · Stochastic methods · Termination rules

Introduction

The global optimization problem is usually defined as:

$$x^* = \arg \min_{x \in S} f(x), \quad (1)$$

with S :

$$S = [a_1, b_1] \otimes [a_2, b_2] \otimes \dots [a_n, b_n],$$

where the function f is assumed to be continuous and differentiable. There is a wide range of problems in the relevant literature that can be treated as global optimization problems. In the area of physics, for example, Yang et al. proposed a multiobjective genetic algorithm on an accelerator lattice [1], Iuliano proposed global optimization techniques for benchmark aerodynamic cases and Duan et al.

used global optimization techniques for conceptual rainfall-runoff models [2]. In the area of chemistry, Heils and Johnston provided a review on the usage of global optimization techniques on clusters [3], Shin et al. proposed a software that utilizes global optimization methods for protein–ligand docking [4], and Liwo et al. proposed a global optimization algorithm for protein structure prediction by minimizing the potential energy function [5]. In addition, in the area of economic sciences, Gaing proposed a particle swarm optimization method to solve the economic dispatch [6] and Maranas et al. proposed global optimization methods for tackling long-term financial planning problems [7]. Furthermore, in medicine, Lee proposed large-scale optimization-based classification models applied to medical problems [8] and Cherruault reviewed some global optimization methods [9].

There are several ways to categorize global optimization techniques, but most researchers suggest dividing them into two broad categories: deterministic and stochastic techniques. In the area of deterministic techniques, the one with the greatest spread among researchers is the interval method [10, 11], where the initial interval of values S of

✉ Ioannis G. Tsoulos
itsoulos@uoi.gr

¹ Department of Informatics and Telecommunications,
University of Ioannina, Arta, Greece

the objective function is continuously divided into smaller parts. The segments that may not contain the total minimum are discarded and eventually a very narrow interval of values should be left which will contain the total minimum of the objective function. Also, in the area of deterministic methods, many related works have been presented with various applications, such as the work of Maranas and Floudas that proposed a deterministic method for molecular structure determination [12], the TRUST method for global optimization [13], the method of Evtushenko and Posypkin for global box-constrained optimization [14], a method that uses smooth diagonal auxiliary functions to approximate the global minimum [15], etc. Also, Kunde et al. [16] suggested the usage of deterministic global optimization methods for chemical problems. Furthermore, recently, Sergeyev et al. [17] proposed a systematic comparison between deterministic and stochastic methods used to handle the global optimization problem. Recently, Yassami and Ashtari proposed a hybrid optimization method to handle the optimization problems [18].

On the other hand, in the category of stochastic techniques, where the greatest research effort is made, techniques are presented that look for the total minimum using stochastic methods, which, of course, does not guarantee finding the total minimum. In this vast area of research endeavors, one encounters methods such as controlled random search methods [19–21], which can be considered as a direct search method, simulated annealing methods [22–24], differential evolution methods [25, 26], particle swarm optimization methods [27–29], ant colony optimization [30, 31], and genetic algorithms [32–34]. In addition, since in recent years there has been an extremely wide spread of parallel computing units, many studies have appeared that utilize the modern parallel processing units [35–37] to tackle the global optimization problem. Some research that one can study regarding metaheuristic algorithms is presented in some recent papers [38–40]. This paper suggests a number of directions for efficient parallelization of particle swarm optimization (PSO) techniques.

Parallel optimization methods can be utilized in various application domains, enabling rapid resolution of complex problems. Some of these domains include:

1. Engineering and design: optimizing parameters for improved performance in engineering systems like aircraft, automobiles, and machinery.
2. Computer science: optimizing complex algorithms and computational tasks.
3. Biology and pharmaceuticals: addressing problems like DNA analysis and calculating multiple protein interactions.
4. Economics and finance: automating the optimization of intricate models.

5. Energy optimization: optimizing energy consumption in energy networks.
6. Climate change: optimizing complex climatic models and weather prediction.
7. Telecommunications: designing efficient communication networks and optimizing network performance and bandwidth management.
8. Policy decision-making: finding optimal solutions for problems like transportation routing and sustainability policies.

By applying parallel optimization techniques in these diverse fields, researchers and practitioners can efficiently tackle complex challenges and achieve optimal solutions more effectively.

The PSO method is inspired by the observations of Eberhart and Kennedy in the 1990s. The two observed in detail the behavior of birds as they move through space looking for food and then formulated a technique that falls under the domain of global optimization, using the experience they gained from their observations. In the new technique they formulated, the atoms or particles move in the research space of the problem in search of the minimum of the objective function. These particles are directly related to two basic characteristics: the current position of each particle and the velocity at which it moves in search of the global minimum. The current position is denoted as $\vec{x}$ and the velocity is referred to as $\vec{v}$. In addition, for each particle, the best position in which it has been found in the past (the one with the smallest value for the objective function) is kept in memory, but also, for the total population, its best position is kept. The method moves the particles to search for the global minimum through an iterative process, in which the positions of the particles at each iteration are derived from each particle's current position, the best position it has found in previous iterations, and the overall best position of the population.

Due to the simplicity of the method and the small number of parameters that should be set, this method has been applied to many difficult problems from all areas of the sciences, such as problems that arise in physics [41, 42], chemistry [43, 44], medicine [45, 46], and economics [47]. Also, the PSO method was successfully applied recently to many practical problems such as flow shop scheduling [48], successful development of electric vehicle charging strategies [49], emotion recognition [50], and robotics [51]. An extensive tutorial on PSO methods can be found in the work of Marini and Walczak [52]. Moreover, a recent overview of PSO variants can be found in the work of Jain et al. [53]. Since the technique has gained a lot of popularity in recent years, many works have been developed based on it, such as combination with the mutation mechanism [54–56], methods that propose new techniques to initialize the velocity vector [57], hybrid techniques [58–60], and parallel techniques

[61–63]. Also, since the calculation of the velocity of the particles is a determining factor for the effectiveness of the method, several techniques were formulated for the calculation of the velocity [64–66]. The method of particle swarm optimization has been successfully combined with other global optimization techniques, such as the work of Bogdanova et al. [67], who suggested a combination of grammatical evolution with PSO [68]. Also, Pan et al. [69] suggested a method that combines the PSO method with a variation of simulated annealing. Likewise, Mughal et al. [70] proposed a hybrid technique that effectively combines PSO and simulated annealing for solar energy problems. In the same way, Lin et al. [71] suggested a hybrid method of PSO and differential evolution for some optimization problems. Additionally, a number of techniques have been presented in which local minimization methods are applied to some iterations of particle swarm optimization [72, 73]. Also, Gao et al. suggested the usage of a PSO method to reduce energy consumption [74]. Of course, these techniques significantly increase the computational time required by the method, but also significantly improve the performance of the method.

Since the topic of global optimization is very widely applied in many fields but also due to the fact that it requires significant computing resources, techniques that take advantage of parallel architectures have to be developed to reduce the required computing time. For example, Olenšek et al. [75] proposed an asynchronous parallel global optimization method based on simulated annealing and differential evolution applied on a series of benchmark global optimization problems. Also, Regis and Shoemaker [76] employed radial basis functions [77] for parallel global optimization. A systematic review of parallel metaheuristics can be found in the work of Alba et al. [78]. Also, a review of metaheuristics that are implemented on modern GPU architectures can be found in the work of Essaid et al. [79]. The particle swarm optimization method is an excellent candidate method for parallelization, since it is based on a series of computational solutions that partially act independently of each other. For example, Koh et al. proposed a parallel asynchronous implementation of the PSO algorithm [80], Tewolde et al. [81] proposed a multi-swarm PSO algorithm where the swarm is divided into a set of subswarms that are executed in parallel. Also, Ouyang et al. [82] proposed a parallel implementation of the PSO algorithm using the CUDA programming library to solve the 1D heat conduction equation. Furthermore, Campos et al. [83] investigated the impact of communication strategies on a parallel PSO implementation used for solving many objective optimization problems. A survey of parallel PSO algorithms can be found in the work of Lalwani et al. [84].

In the present work, the initial population of particle swarm optimization is divided into independent populations running on different parallel computing units, which will also be called islands. In addition, a number of techniques

are proposed, which aim at efficient communication between the independent islands on the one hand, and at the early termination of the overall method on the other. These techniques include a new method of calculating particle velocity, a new termination rule specifically modified for parallel techniques, and a new way of propagating the best particles among the parallel computing units involved in the overall method. The proposed velocity calculation is proposed by Eberhart and Shi [85] and it is based on random values. Also, the proposed stopping rule is based on the work of Charillogis and Tsoulos [86], giving excellent results when applied on a series of well-known optimization problems. However, the proposed termination technique is modified appropriately for parallel computing environments.

The main disadvantages of the PSO method are its tendency to converge quickly to a local minimum in some complex or high-dimensional objective functions, and its performance heavily depends on the initial particle distribution, as incorrect values may lead to suboptimal solutions. The proposed method overcomes these issues and aims to improve the algorithm's performance in terms of speed and reduced consumption of computational resources. In general, any optimization method can be parallelized, ensuring the ability to utilize fewer computational resources and saving time in the resolution process.

The rest of this article is organized as follows: in “[The Proposed Method](#)” the proposed method and the new approaches in particle swarm optimization are discussed in detail, in “[Experiments](#)”, the used test functions as well the experimental results are fully outlined and finally in “[Conclusions](#)” some conclusions and future guidelines are listed.

The Proposed Method

This section will begin with a detailed description of the serial method to be performed on each parallel processing unit as well as the overall parallel method proposed in this work. Subsequently, the basic components of the proposed process, such as the calculation of the velocity, the proposed termination rule, and the method of propagating points between the parallel computing units will be thoroughly analyzed.

In every parallel processing unit, a PSO algorithm is executed to locate the global minimum of the objective function. The formulation of the PSO algorithm is shown in Algorithm 1. The approach of the underlying algorithm is based on representing the problem as a swarm of particles, where each particle represents a candidate solution in the Euclidean search space. During the optimization process, the particles move in the search space based on their individual experience and also on social interactions. Using these two influences, the particles adjust their velocities and positions during the iterations of the

algorithm. The parallel version of the underlying article also promotes social interactions among sub-swarms using different propagation schemes.

p_i and p_{best} . A typical computation of the new velocity is defined as:

$$u_i = \omega u_i + r_1 c_1 (p_i - x_i) + r_2 c_2 (p_{\text{best}} - x_i) \quad (3)$$

Algorithm 1 The base PSO algorithm executed in one processing unit.

1. **Initialization Step** .

- (a) **Set** iter = 0.
- (b) **Set** m as the total number of particles.
- (c) **Set** iter_{max} as the maximum number of iterations allowed for the serial algorithm.
- (d) **Initialize** randomly, the positions $x_1, x_2, \dots, x_m$ for the particles. The initial positions must be within the domain of the objective function.
- (e) **Initialize** randomly the velocities $u_1, u_2, \dots, u_m$.
- (f) **For** $i = 1..m$ do $p_i = x_i$. The vector p_i represents the best located position of particle i .
- (g) **Set** $p_{\text{best}} = \arg \min_{i \in 1..m} f(x_i)$

2. **Termination Check Step** . The termination criterion for the serial algorithm is checked. If the termination criterion is true, then the method terminates.

3. **For** $i = 1..m$ **Do**

- (a) **Compute** the velocity u_i as a combination of the vectors u_i , p_i and p_{best}
- (b) **Set** the new position for the particle as: $x_i = x_i + u_i$
- (c) **Calculate** the $f(x_i)$ for particle x_i using the objective function $f(x)$.
- (d) **If** $f(x_i) \leq f(p_i)$ then $p_i = x_i$

4. **End For**

5. **Set** $p_{\text{best}} = \arg \min_{i \in 1..m} f(x_i)$

6. **Set** iter = iter + 1.

7. **Goto** Step 2

The base PSO algorithm, as described in Algorithm 1, computes at every iteration the new position of the particle i using the following operation:

$$x_i = x_i + u_{i+1}. \quad (2)$$

In most cases, the new velocity could be a linear combination of the previously computed velocity and the best values

where

- 1. The variables r_1 , r_2 are random numbers defined in $[0, 1]$.
- 2. The constants c_1 , c_2 are defined in $[1, 2]$.

3. The variable ω is called inertia and it was suggested by Shi and Eberhart [27]. In the current article, a simple inertia calculation is used and calculated using:

$$\omega_{\text{iter}} = 0.5 + \frac{r}{2}. \quad (4)$$

The variable r is a random number with $r \in [0, 1]$.

The Parallel Algorithm

The overall parallel algorithm, which runs on N_f independent computing units, is shown in Algorithm 2.

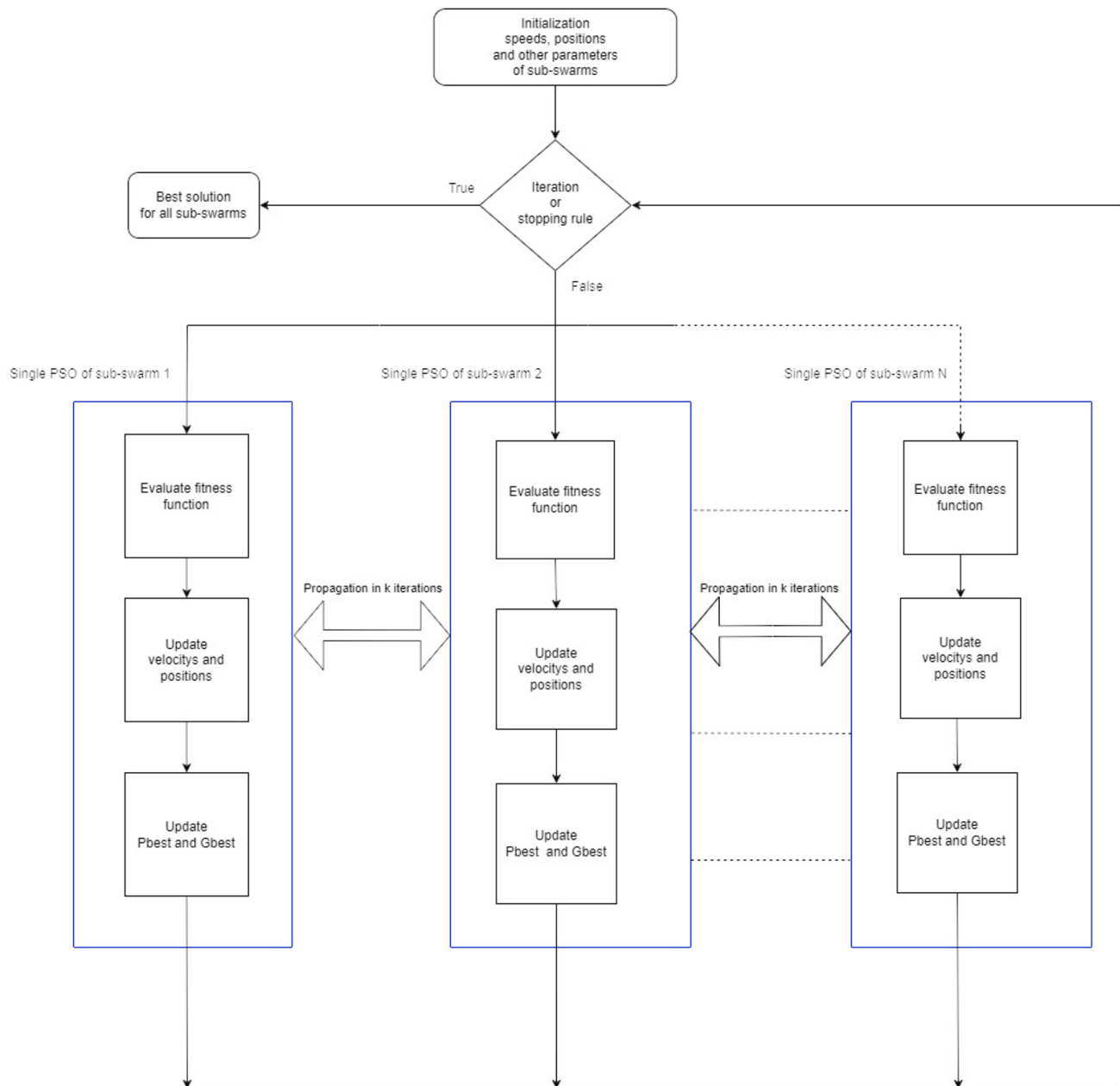
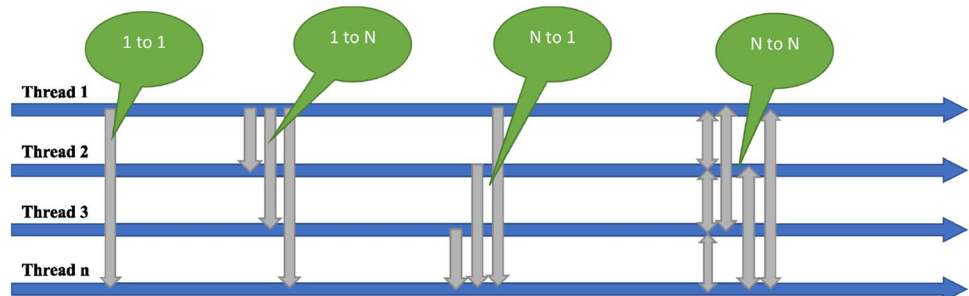


Fig. 1 The overall algorithm

Algorithm 2 The implemented parallel algorithm.

1. **Set** as N_I the total number of parallel processing units.
2. **Set** as N_R as the number of iterations, after which each processing unit will send its best particles to the remaining processing units.
3. **Set** N_P the number of migrated particles between the parallel processing units.
4. **Set** $K = 0$ the iteration number.
5. **For** $j = 1, \dots, N$ do in parallel
 - (a) **Execute** an iteration of the PSO algorithm described in algorithm 1 on processing unit j .
 - (b) **If** $K \bmod N_R = 0$, then
 - i. **Get** the best N_P particles from algorithm j .
 - ii. **Propagate** these N_P particles to the rest of processing units using some propagation scheme that will be described subsequently.
 - (c) **EndIf**
6. **End For**
7. **Update** $K = K + 1$
8. **Check** the proposed termination rule. If the termination rule is valid, then goto step 8a else goto step 5.
 - (a) **Terminate** and report the best value from all processing units.. Apply a local search procedure to this located value to enhance the located global minimum. In the proposed algorithm a BFGS variation of Powell [88] was used as a local search procedure.

Fig. 2 A graphic presentation of all propagation schemes



The two main components of the proposed parallel implementation are the particle propagation mechanism between the parallel computing units and the proposed termination rule. In the first case, it must be decided which units will send the best particles to other parallel units. Units receiving particles replace their worst members with the ones they receive. In the second case, a termination rule based on asymptotic observations is proposed for early termination of the parallel technique. In the proposed termination technique, the overall algorithm is terminated if it holds with some certainty if some asymptotic termination criterion holds for at least one parallel computing unit. These components will be discussed in the following subsections. The overall algorithm is graphically outlined in Fig. 1.

Propagation Mechanism

During the execution of the parallel algorithm and periodically the processing units propagate their best particles (those with the lowest function value) to the remaining processing units. This propagation can be achieved in the following possible ways:

1. **1 to 1.** In this propagation scheme, a randomly selected processing unit will send to some other randomly selected unit its N_p best particles.
2. **1 to N.** During this scheme, a randomly selected unit will send its best N_p particles to the remaining units.
3. **N to 1.** In this scheme, all processing units will send the corresponding N_p best particles of each unit to a randomly selected unit.
4. **N to N.** For this scheme, all processing units will send the corresponding N_p best particles to all.

All propagation schemes are demonstrated graphically in Fig. 2.

Stopping Rule

In the proposed technique, a distinct termination rule is checked on each parallel processing unit. This rule is formulated as follows:

$$\delta_i^{(k)} = \left| f_{i, \min}^{(k)} - f_{i, \min}^{(k-1)} \right|. \quad (5)$$

This quantity is calculated on every iteration k . The value $f_{i, \min}^{(k)}$ is the best function value for unit i at iteration k . If the above quantity is less than a predetermined limit ϵ for N_M continuous repetitions, then the algorithm executed on this unit is terminated. In the present work, if a parallel processing unit is terminated, then the overall process is also terminated.

Experiments

To be able to establish the efficiency of the method in finding the total minimum of continuous functions, a series of experiments were performed in which objective functions from the relevant literature were used [88, 89]. These objective functions have been extensively studied by many researchers in many publications [90–93]. In this series of experiments, beyond the ability of the method to find the global minimum, the effect of changes in a number of critical parameters, such as propagation techniques, on the behavior of the proposed method was also evaluated. In addition, experiments were conducted in which the actual execution time was measured for various objective functions. In these experiments, the actual reduction in execution time was also recorded as the number of parallel processing units increased.

Test Functions

The test functions used in the conducted experiments are listed in Table 1.

Experimental Results

Experiments were performed on the objective functions presented previously. Each experiment was run 30 times using different random numbers each time and the averages of the experiments are plotted in the tables below. For the parallelization the freely available OpenMP library [96] was utilized. The programming language used was ANSI C++ and the OPTIMUS optimization package freely available from <https://github.com/itsoulos/OPTIMUS> (accessed on 3 March 2023) was used. The most critical parameters of the proposed method are listed in Table 2.

In the first set of experiments, the serial version of the proposed technique, i.e., running on a single processor, was compared with the following widely used global optimization techniques:

1. A genetic algorithm, denoted as GENETIC in the the following table.
2. A simple version of particle swarm optimization, denoted as PSO in the following table.
3. The parallel implementation of the GaLib library [97], denoted as GALIB in the experimental results.
4. The parallel global best–worst particle swarm optimization algorithm [98], denoted as PGBWPSO.

In all cases, the same number of particles or chromosomes and the same maximum number of generations were used. In addition, after each method was executed, the local optimization method BFGS was used to improve the global

Table 1 The objective functions used in the experiments

Name	Formula	Dimension
<i>Cigar function</i>	$f(x) = x_1^2 + 10^6 \sum_{i=2}^n x_i^2$	$n = 10$
Bf1	$f(x) = x_1^2 + 2x_2^2 - \frac{3}{10} \cos(3\pi x_1) - \frac{4}{10} \cos(4\pi x_2) + \frac{7}{10}$	$n = 2$
Bf2	$f(x) = x_1^2 + 2x_2^2 - \frac{3}{10} \cos(3\pi x_1) \cos(4\pi x_2) + \frac{3}{10}$	$n = 2$
Branin	$f(x) = \sum_{i=1}^n x_i^2 - \frac{1}{10} \sum_{i=1}^n \cos(5\pi x_i)$	$n = 2$
CM4	$f(x) = \sum_{i=1}^n x_i^2 - \frac{1}{10} \sum_{i=1}^n \cos(5\pi x_i)$	$n = 4$
Camel	$f(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 4x_2^4$	$n = 2$
Discus10	$f(x) = 10^6 x_1^2 + \sum_{i=2}^n x_i^2$	$n = 10$
Easom	$f(x) = -\cos(x_1) \cos(x_2) \exp((x_2 - \pi)^2 - (x_1 - \pi)^2)$	$n = 2$
Exp	$f(x) = -\exp(-0.5 \sum_{i=1}^n x_i^2), \quad -1 \leq x_i \leq 1$	$n = 4, 16, 64$
Griewank2	$f(x) = 1 + \frac{1}{200} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \frac{\cos(x_i)}{\sqrt{ i }}$	$n = 2$
Gkls[94]	$f(x) = \text{Gkls}(x, n, w)$	$n = 2, 3$
Hansen	$f(x) = \sum_{i=1}^5 i \cos[(i-1)x_1 + i] \sum_{j=1}^5 j \cos[(j+1)x_2 + j]$	$n = 2$
Hartman 3	$f(x) = -\sum_{i=1}^4 c_i \exp(-\sum_{j=1}^3 a_{ij}(x_j - p_{ij})^2)$	$n = 3$
Hartman 6	$f(x) = -\sum_{i=1}^4 c_i \exp(-\sum_{j=1}^6 a_{ij}(x_j - p_{ij})^2)$	$n = 6$
Potential[95]	$V_{LJ}(r) = 4e \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$	$n = 3, 5$
Rastrigin	$f(x) = x_1^2 + x_2^2 - \cos(18x_1) - \cos(18x_2)$	$n = 2$
<i>Rosenbrock</i>	$f(x) = \sum_{i=1}^{n-1} (100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2)$	$n = 4, 8$
Shekel 5	$f(x) = -\sum_{i=1}^5 \frac{1}{(x-a_i)(x-a_i)^2 + c_i}$	$n = 4$
Shekel 7	$f(x) = -\sum_{i=1}^7 \frac{1}{(x-a_i)(x-a_i)^2 + c_i}$	$n = 4$
Shekel 10	$f(x) = -\sum_{i=1}^{10} \frac{1}{(x-a_i)(x-a_i)^2 + c_i}$	$n = 4$
Sinu	$f(x) = -(2.5 \prod_{i=1}^n \sin(x_i - z) + \prod_{i=1}^n \sin(5(x_i - z)))$	$n = 4, 8$
Test2n	$f(x) = \frac{1}{2} \sum_{i=1}^n x_i^4 - 16x_i^2 + 5x_i$	$n = 4, 5, 6, 7$
Test30n	$\frac{1}{10} \sin^2(3\pi x_1) \sum_{i=2}^{n-1} ((x_i - 1)^2 (1 + \sin^2(3\pi x_{i+1}))) + (x_n - 1)^2 (1 + \sin^2(2\pi x_n))$	$n = 3, 4$

Table 2 The values for most critical parameters of the algorithm

Parameter	Meaning	Value
m	Number of particles	200
$\text{iter}_{\max}$	Maximum number of generations	200
c_1	Cognitive parameter for PSO	1.0
c_2	Social learning parameter for PSO	1.0
N_R	The particle exchange frequency	15
ϵ	Small value used in termination rule	10^{-6}
N_M	Number of repetitions used in termination rule	15

minimum found. The results for this experiment are reported in Table 3.

In the table, each number in each cell represents the average of the function calls for 30 runs. Also, the numbers in parentheses stand for the percentage of runs in which the global minimum was successfully found. If this percentage

is not present, it implies 100% success. In addition, a line has been added at the end of the table showing the total number of function calls for each method. The experimental results demonstrate that even running the proposed technique on a single computing unit can produce better results than the other techniques in terms of the required number of function calls, and, by extension, the time required to find the global minimum. Moreover, although method PGBWPSO requires less total number of calls than the proposed technique, the proposed technique is shown to significantly outperform it in its reliability of finding the global minimum.

To evaluate the effect of executing the method on parallel processing units, another experiment was done in which the number of parallel processing units was increased from one to ten and the results are presented in Table 4. Also, a box plot for this experiment is shown in Fig. 3. Additionally, a Friedman test was conducted to determine whether there were significant differences in the scores of function calls on five ($N_I = 1, 2, 4, 5$, and 10) repeated number of

Table 3 Comparison of the proposed method against two other global optimization techniques

Function	GENETIC	PSO	GALIB	PGBWPSO	PROPOSED
BF1	9581	19,144 (0.83)	11,802	1988 (0.16)	12,625
BF2	10,014	19,121 (0.90)	11,549	1980 (0.36)	13,108
BRANIN	9289	17,760	9003	1780 (0.93)	8574
CIGAR10	40,226	39,553	28,498 (0.50)	16,649	40,274
CM4	15,360	22,829	1513 (0.70)	9169 (0.20)	11,512
DISCUS10	40,216	22,359	28,798	11,342	37,848
EASOM	9994	2897	35,550	1381	4608
ELP10	40,273	31,192	29,654	8993	23,436
EXP4	14,084	21,375	8349	11,033	9062
EXP16	40,215	27,755	8800	24,885	22,408
EXP64	40,237	26,155	9455	38,800	40,238
GKLS250	8361	16,217	12,645	1906 (0.83)	8070
GKLS350	12,697 (0.97)	20,393	15,351 (0.97)	6093 (0.80)	10,696 (0.97)
GRIEWANK2	9298 (0.97)	20,004 (0.87)	11,344 (0.67)	1544 (0.13)	11,064
POTENTIAL3	27,799	20,876	15,456	13,054	12,876
POTENTIAL5	40,240	25,809	15,902	13,292	38,377
HANSEN	14,951 (0.93)	16,945	35,011	1305 (0.83)	12,467
HARTMAN3	11,268	22,259	35,211	5916 (0.96)	10,018
HARTMAN6	21,396 (0.63)	33,679 (0.33)	60,457 (0.60)	11,261 (0.43)	15,082 (0.53)
RASTRIGIN	8967	16,044	22,753 (0.97)	2138 (0.36)	9286 (0.93)
ROSENBROCK4	40,233	26,367	15,701	11,447	25,120
ROSENBROCK8	40,271	32,750	18,449	17,212	38,577
SHEKEL5	19,403 (0.70)	29,079 (0.33)	50,453	8658 (0.16)	15,409 (0.43)
SHEKEL7	19,376 (0.80)	27,817 (0.47)	60,791 (0.77)	7839 (0.13)	14,989 (0.63)
SHEKEL10	19,829 (0.77)	26,479 (0.83)	61,450 (0.77)	7693 (0.23)	15,087 (0.67)
SINU4	15,788	23,915	60,805	10,367 (0.43)	12,298
SINU8	30,928	27,834 (0.97)	90,300	13,307 (0.56)	15,500
TEST2N4	17,109	23,983 (0.97)	20,844	8625 (0.70)	14,520 (0.70)
TEST2N5	19,464	30,817	20,751 (0.87)	9615 (0.43)	14,801 (0.47)
TEST2N6	24,217	29,067 (0.90)	20,898 (0.80)	9185 (0.20)	17,444 (0.23)
TEST2N7	26,824	32,337 (0.60)	32,553 (0.60)	10,007 (0.20)	22,780 (0.23)
TEST30N3	17,575	15,660	16,857	5061 (0.96)	7814
TEST30N4	17,395	23,519	16,900	6983 (0.96)	8014
Total	732,878 (0.96)	791,990 (0.91)	894,303 (0.92)	310,508 (0.67)	573,980 (0.87)

The number of processing units is set to $N_f = 1$

units and the results are shown in Fig. 4. In addition, to give greater reliability to the experimental results, the total number of particles was in all runs constant and equal to 200, as defined in the Table 2. This means, for example, that when the computing units are four, each unit will run a serial version of PSO with 50 particles, and when the computing units become five, the number of particles in each unit will be reduced to 40.

From the experimental results, it is clear that increasing the number of parallel processing nodes further reduces the total number of function calls, since more units are used in the attempt to find the global minimum. In addition, a slight

increase in the reliability of the method in finding the total minimum is also observed.

Furthermore, to determine the effect of the propagation mechanism on the reliability and speed of the method, another comparative experiment was performed, in which the number of parallel processing units was set to 5 ($N_f = 5$) and all propagation mechanisms were used. The results of this experiment are presented in Table 5.

This time, the experimental results did not show any clear superiority of one of the propagation methods. However, it appears that the 1-to-N propagation method has slightly better performances than the rest of the best particle propagation techniques among the sub-populations.

Table 4 Experimental results using the proposed method, the propagation scheme was set to 1to1 and the value of N_p was set to 5. In the conducted experiments, the number of parallel processing units was varied from 1 to 10

Function	$N_l = 1$	$N_l = 2$	$N_l = 4$	$N_l = 5$	$N_l = 10$
BF1	12,625	11,660	9984	10,315	6667
BF2	13,108	11,600	10,363	9403	6964
BRANIN	8574	6953	5412	5170	4141
CIGAR10	40,274	40,180	39,763	38,887	21,291
CM4	11,512	12,019	12,203	12,339	9910
DISCUS10	37,848	26,044	13,211	10,989	4171
EASOM	4608	3927	3660	3513	3110
ELP10	23,436	26,469	14,268	11,100	7462
EXP4	9062	9691	9678	9556	9431
EXP16	22,408	15,608	18,025	21,307	21,991
EXP64	40,238	40,177	39,856	39,731	24,234
GKLS250	8070	7809	7225	6853	5591
GKLS350	10,696 (0.97)	11,488	10,578	10,095	7279
GRIEWANK2	11,064	10,681	9127	8926	5604
POTENTIAL3	12,876	5568	5018	4756	4333
POTENTIAL5	38,377	4905	4455	4221	4016
HANSEN	12,467	5067	4340	4031	3518
HARTMAN3	10,018	10,263	10,162	9711	8234
HARTMAN6	15,082 (0.53)	9816 (0.73)	7212 (0.97)	7194	5935
RASTRIGIN	9286 (0.93)	9432	6227	5974	4254
ROSENBROCK4	25,120	20,084	16,454	12,244	7574
ROSENBROCK8	38,577	25,195	22,531	18,508	9587
SHEKEL5	15,409 (0.43)	14,112 (0.77)	8575 (0.87)	7898 (0.93)	4948
SHEKEL7	14,989 (0.63)	13,800 (0.90)	9227 (0.93)	8717 (0.97)	5050
SHEKEL10	15,087 (0.67)	14,662 (0.87)	10,268 (0.93)	8229 (0.93)	4871 (0.97)
SINU4	12,298	12,997	13,172	12,842	10,316
SINU8	15,500	15,475	16,442	16,375	12,732
TEST2N4	14,520 (0.70)	15,043 (0.87)	12,346	9769	4566
TEST2N5	14,801 (0.47)	16,097 (0.77)	12,358 (0.90)	9440 (0.93)	4813 (0.93)
TEST2N6	17,444 (0.23)	16,224 (0.47)	9103 (0.73)	7855 (0.57)	4410 (0.53)
TEST2N7	22,780 (0.23)	20,330 (0.47)	12,699 (0.40)	9773 (0.50)	4243 (0.47)
TEST30N3	7814	7967	7010	6382	5014
TEST30N4	8014	7683	6568	6317	5090
Total	573,980 (0.87)	479,026 (0.96)	397,519 (0.96)	368,460 (0.96)	251,350 (0.97)

The last experiment had to do with the effect of the N_p parameter on the speed of the method. In it, five parallel computing units were used and the propagation method was set to **Nto1**. The experimental results are presented in Table 6.

As is evident, increasing the value of the N_p parameter significantly reduces the required number of function calls up to a value of 5. From then on, the total number of calls appears to be constant. Practically, this means that sending not only the best particle, but also those with the immediately lower value further reduces the required number of function calls.

Experiments for the Execution Time

To understand the efficiency of the method with respect to execution time, three additional series of experiments were performed. In all experiments, the total number of particles in the particle swarm optimization was fixed at 500 and from 1 to 20 parallel processing units were used, and the total execution time was measured for 30 different independent runs. The experiments were conducted on AMD Epyc 7552 equipped with 32 GB of RAM. The operating system was the Ubuntu 20.04.

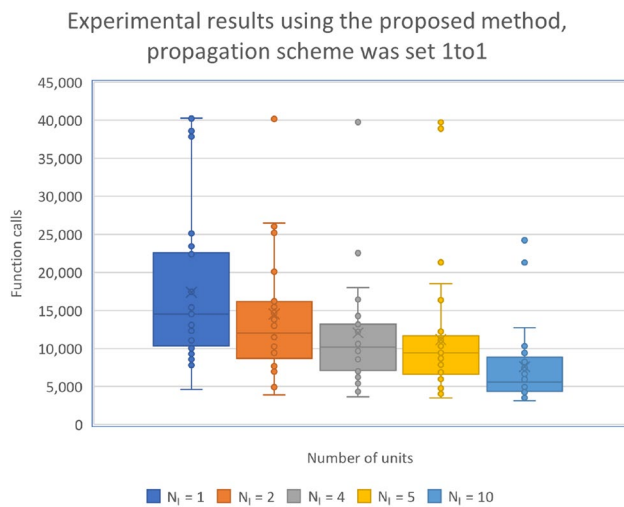


Fig. 3 Box plot for the comparison between different number of processing units. The propagation method was set to 1to1

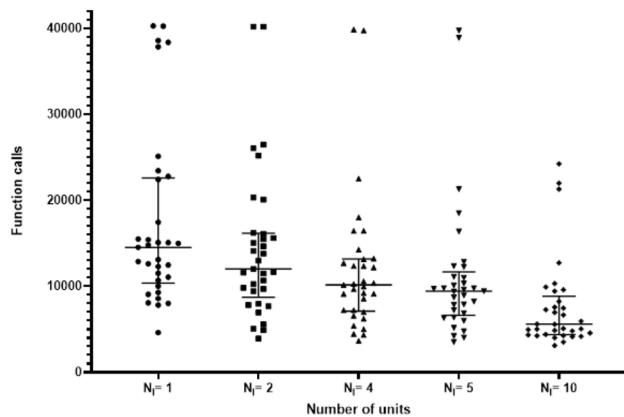


Fig. 4 We conducted a Friedman test to determine whether there were significant differences in the scores of function calls on five ($N_i = 1, 2, 4, 5$, and 10) repeated number of units. The Friedman Chi-square was 94.38 with 4 degrees of freedom, indicating a significant difference between the groups ($p < 0.0001$)

During the first series of the additional experiments, the high conditioned elliptic function, defined as

$$f(x) = \sum_{i=1}^n (10^6)^{\frac{i-1}{n-1}} x_i^2,$$

was used as a test case. In the experiments, the values $n=10, 20, 30, 40, 50$ were used and the experimental results are displayed graphically in Fig. 5.

From the experimental results, it follows that doubling the parallel processing units almost halves the required execution time in most cases.

During the second series of experiments, different versions of the GKLS function were used for dimensions from two to five. This function was thoroughly tested also in the work of Sergeyev et al. [17]. In each dimension, ten different instances of the function were used to have a safe estimate of both the performance of the method and the possible decrease in execution time as the parallel processing units. The average execution time for different dimensions of the GKLS function and for different numbers of processing units is graphically shown in Fig. 6.

As is shown, the execution time decreases rapidly with increasing parallel processing units in all instances of the function. Furthermore, the average number of function calls for every case is also measured and the results are plotted in Fig. 7.

Increasing the number of parallel processing units also reduces the number of required function calls, especially for the low-dimensional cases (Gkls2 and Gkls3). However, the most interesting point of this series of experiments is the ability of the proposed technique to locate the global minimum as the number of parallel computing units increases. This capability is graphically represented in Fig. 8.

From this graph, it can be concluded that the increase in parallel processing units significantly improves the efficiency of the technique in finding the global minimum.

In the third series of additional experiments, a more difficult experiment was used, but one that has multiple applications. The proposed method was used to train an artificial neural network [99, 100] with ten processing nodes applied on two widely used classification problems from the relevant literature: the Wine problem [101, 102] and the Parkinsons problem [103]. The results are graphically outlined in Fig. 9.

Again, the required training time decreases drastically with increasing parallel processing units.

Conclusions

In the present work, a number of ideas were presented that can lead to an efficient parallel implementation of the PSO technique. The need for parallelization of the method is great, as it is an iterative process involving repeated operations on vectors and these operations for large problems may require significantly large computational time. In the current work, a technique of propagating the best particles between computing units as well as a termination rule of the overall process was introduced. In the case of the propagation of the best particles from the experiments carried out, it appears that it is more efficient to send between the computing units more than the best particle. Furthermore, the propagation method between parallel computing units did not have a drastic effect on the efficiency and speed of the method, although the 1-to-N propagation method appeared

Table 5 Comparison of different propagation schemes

Function	1to1	1toN	Nto1	NtoN
BF1	10,315	8408	9471	8647
BF2	9403	8024	10,578	7730
BRANIN	5170	4633	5203	5789
CIGAR10	38,887	25,035	35,527	34,258
CM4	12,339	14,195	12,296	12,565
DISCUS10	10,989	6484	7667	6154
EASOM	3513	3072	3496	3121
ELP10	11,100	13,027	9598	7091
EXP4	9556	10,654	9517	10,607
EXP16	21,307	29,289	23,833	27,307
EXP64	39,731	11,959	38,191	26,175
GKLS250	6853	6568	6966	7808
GKLS350	10,095	10,366	10,400	9674
GRIEWANK2	8926	6022	7791	5432
POTENTIAL3	4756	4011	4591	4075
POTENTIAL5	4221	4002	4176	3870
HANSEN	4031	3092	4320	3265
HARTMAN3	9711	10,154	9316	11,110
HARTMAN6	7194	6914 (0.73)	6760 (0.97)	11,242 (0.73)
RASTRIGIN	5974	4077	5383	4804
ROSENBROCK4	12,244	13,930	11,511	14,710
ROSENBROCK8	18,508	15,423	16,659	20,848
SHEKEL5	7898 (0.93)	9604 (0.90)	7513 (0.93)	10,657 (0.67)
SHEKEL7	8717 (0.97)	12,204	9404	10,805 (0.70)
SHEKEL10	8229 (0.93)	13,418 (0.93)	9693	12,677 (0.83)
SINU4	12,842	14,757	13,154	13,376
SINU8	16,375	22,754	17,026	20,121
TEST2N4	9769	6633 (0.97)	10,289	7483 (0.83)
TEST2N5	9440 (0.93)	4819 (0.93)	8077 (0.90)	5429 (0.80)
TEST2N6	7855 (0.57)	5358 (0.77)	8354 (0.60)	5574 (0.43)
TEST2N7	9773 (0.50)	5183 (0.33)	7417 (0.53)	6312 (0.40)
TEST30N3	6382	6538	6176	7462
TEST30N4	6317	6938	6473	6305
Total	368,460 (0.96)	327,545 (0.96)	356,826 (0.97)	352,483 (0.92)

The number of processing units was set to 5

to have slightly better results. Furthermore, the increase of the parameter N_p from one to five decreased the required number of function calls. However, the biggest gain from using the method lies in the increase in parallel processing units. From the experiments performed, it is evident that as parallel processing units increase, the total function calls required to find the global minimum decreases almost for every objective function. In addition, the increase in parallel processing units improved to some extent the efficiency of the method in finding the global minimum. The effectiveness of the proposed method was also shown in the comparison between its serial version and two other widely available techniques in the relevant literature. In this comparison, the

serial version of the method required significantly fewer function calls to find each global minimum. Overall, the main contributions of the proposed method are: the velocity calculation mechanism, the propagation among parallel computing units, and the termination rule representing an innovative technique in improving the performance of the algorithm.

On a percentage basis, the proposed technique significantly outperforms the other two compared global optimization techniques. The serial version of the proposed technique reduces the required number of function calls by 22–28% compared to the genetic algorithm and the particle swarm optimization techniques, respectively. However, with the

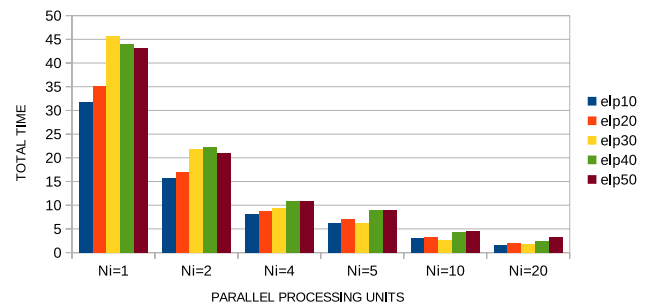
Table 6 The effect of the parameter N_p on the speed of the proposed method

Function	$N_p = 1$	$N_p = 2$	$N_p = 3$	$N_p = 5$	$N_p = 10$
BF1	10,114	9976	10,257	9471	9307
BF2	9224	10,413	10,051	10,578	9530
BRANIN	7037	6027	6238	5203	4851
CIGAR10	39,244	35,793	35,811	35,527	35,835
CM4	11,839	11,588	12,061	12,296	11,878
DISCUS10	8078	6950	9671	7667	11,977
EASOM	3669	3538	3589	3496	3477
ELP10	11,656	12,382	10,643	9598	9641
EXP4	9257	9340	9395	9517	9626
EXP16	27,275	24,008	22,906	23,833	23,190
EXP64	39,679	37,334	32,126	38,191	31,119
GKLS250	7250	6893	7116	6966	6568
GKLS350	10,281	9236	10,088	10,400	9552
GRIEWANK2	9259	10,096	10,297	7791	9527
POTENTIAL3	8471	6694	5770	4591	4829
POTENTIAL5	7127	5869	5301	4176	4844
HANSEN	7978	6230	5591	4320	4573
HARTMAN3	10,162	9939	10,131	9316	10,081
HARTMAN6	10,614	9033	8059	6760	7507
RASTRIGIN	7491	6384	6876	5383	5540
ROSENBROCK4	22,600	16,513	13,631	11,511	12,169
ROSENBROCK8	34,125	23,004	21,027	16,659	19,740
SHEKEL5	12,299	11,923	9521	7513	7256
SHEKEL7	13,895	12,358	10,239	9404	9471
SHEKEL10	14,130	11,536	10,235	9693	6936
SINU4	12,760	12,601	12,268	13,154	11,905
SINU8	16,957	17,327	16,346	17,026	17,030
TEST2N4	12,215	9152	8136	10,289	10,024
TEST2N5	11,384	8429	7934	8077	8935
TEST2N6	11,833	8101	7825	8354	9655
TEST2N7	11,162	8362	8692	7417	9486
TEST30N3	7178	7015	6829	6176	6216
TEST30N4	6518	6676	6509	6473	6756
Total	442,761	390,720	371,169	356,826	359,031

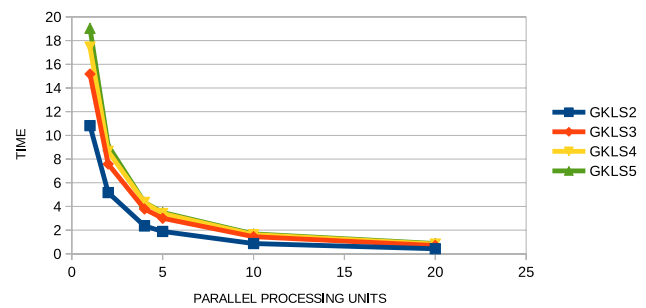
The number of threads was set to five and the value of N_p was changed from one to ten. The propagation scheme used was Nto1

use of more processing units, the reduction in the number of function calls reaches approximately 70%. Furthermore, from the relevant experiments done on the actual execution time, it appears that any doubling of parallel processing threads halves the required execution time required to find the global minimum.

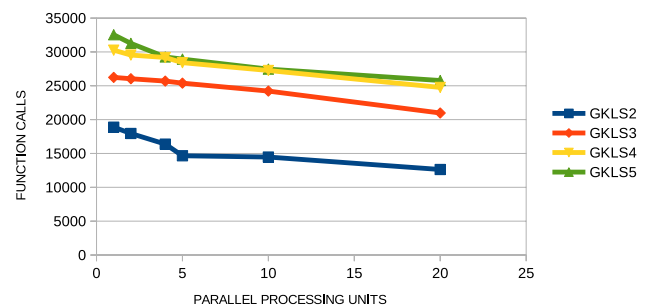
TOTAL EXECUTION TIME FOR ELP

**Fig. 5** Total execution time for different number of parallel processing threads for the objective function ELP

AVERAGE EXECUTION TIME

**Fig. 6** Average time execution for the GKLS function. The dimension of the function varies from two to five

AVERAGE FUNCTION CALLS

**Fig. 7** Average function calls for different cases of the GKLS function

In the future, the important work done in the parallelization of the PSO technique can be continued with the use of new termination rules that will be suitable for execution in parallel computing environments, but also a possible extension of the best particle propagation techniques between parallel computing units.

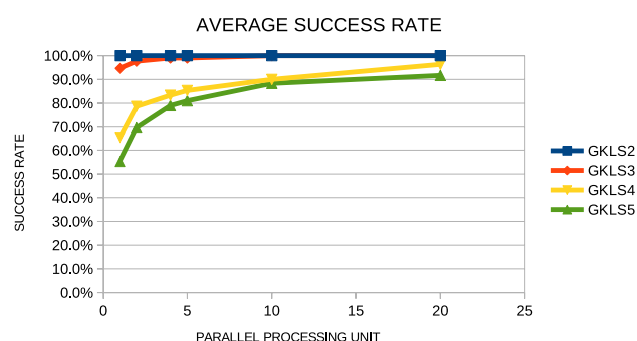


Fig. 8 Average success rate in finding the global minimum for different cases of the Gkls function

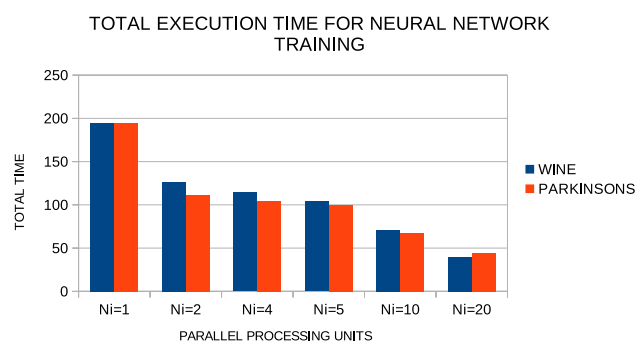


Fig. 9 Total execution time for different number of parallel processing threads for neural network training for two classification datasets

Funding Open access funding provided by HEAL-Link Greece.

Declarations

Conflict of Interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Yang L, Robin D, Sannibale F, Steier C, Wan W. Global optimization of an accelerator lattice using multiobjective genetic algorithms. *Nucl Instrum Methods Phys Res, Sect A*. 2009;609:50–7.
- Duan Q, Sorooshian S, Gupta V. Effective and efficient global optimization for conceptual rainfall-runoff models. *Water Resour Res*. 1992;28:1015–31.
- Heiles S, Johnston RL. Global optimization of clusters using electronic structure methods. *Int J Quantum Chem*. 2013;113:2091–109.
- Shin WH, Kim JK, Kim DS, Seok C. GalaxyDock2: Protein-ligand docking using beta-complex and global optimization. *J Comput Chem*. 2013;34:2647–56.
- Liwo A, Lee J, Ripoll DR, Pillardy J, Scheraga HA. Protein structure prediction by global optimization of a potential energy function. *Biophysics*. 1999;96:5482–5.
- Zwe-Lee G. Particle swarm optimization to solving the economic dispatch considering the generator constraints. *IEEE Trans Power Syst*. 2003;18:1187–95.
- Maranas CD, Androulakis IP, Floudas CA, Berger AJ, Mulvey JM. Solving long-term financial planning problems via global optimization. *J Econ Dyn Control*. 1997;21:1405–25.
- Lee EK. Large-scale optimization-based classification models in medicine and biology. *Ann Biomed Eng*. 2007;35:1095–109.
- Cherruault Y. Global optimization in biology and medicine. *Math Comput Model*. 1994;20:119–32.
- Wolfe MA. Interval methods for global optimization. *Appl Math Comput*. 1996;75:179–206.
- Csendes T, Ratz D. Subdivision direction selection in interval methods for global optimization. *SIAM J Numer Anal*. 1997;34:922–38.
- Maranas CD, Floudas CA. A deterministic global optimization approach for molecular structure determination. *J Chem Phys*. 1994;100:1247.
- Barhen J, Protopopescu V, Reister D. TRUST: a deterministic algorithm for global optimization. *Science*. 1997;276:1094–7.
- Evtushenko Y, Posypkin MA. Deterministic approach to global box-constrained optimization. *Optim Lett*. 2013;7:819–29.
- Sergeyev YD, Kvasov DE. A deterministic global optimization using smooth diagonal auxiliary functions. *Commun Nonlinear Sci Numer Simul*. 2015;21:99–111.
- Kunde C, Michaels D, Micovic J, Lutze P, Górak A, Kienle A. Deterministic global optimization in conceptual process design of distillation and melt crystallization. *Chem Eng Process*. 2016;99:132–42.
- Sergeyev YD, Kvasov DE, Mukhametzhanov MS. On the efficiency of nature-inspired metaheuristics in expensive global optimization with limited budget. *Sci Rep*. 2018;8:453.
- Yassami M, Ashtari PA. Novel hybrid optimization algorithm: dynamic hybrid optimization algorithm. *Multimedia Tools and Applications*; 2023.
- Price WL. Global optimization by controlled random search. *J Optim Theory Appl*. 1983;40:333–48.
- Křivý I, Tvrdík J. The controlled random search algorithm in optimizing regression models. *Comput Stat Data Anal*. 1995;20:229–34.
- Ali MM, Törn A, Viitanen S. A numerical comparison of some modified controlled random search algorithms. *J Global Optim*. 1997;11:377–85.
- Kirkpatrick S, Gelatt CD, Vecchi MP. Optimization by simulated annealing. *Science*. 1983;220:671–80.
- Ingber L. Very fast simulated re-annealing. *Math Comput Model*. 1989;12:967–73.
- Eglese RW. Simulated annealing: a tool for operational research. *Simul Anneal*. 1990;46:271–81.
- Storn R, Price K. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J Global Optim*. 1997;11:341–59.
- Liu J, Lampinen J. A fuzzy adaptive differential evolution algorithm. *Soft Comput*. 2005;9:448–62.

27. Kennedy J, Eberhart R. Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*. 1995;4:1942–8. <https://doi.org/10.1109/ICNN.1995.488968>.
28. Poli R, James Kennedy J, Kennedy J, Blackwell T. Particle swarm optimization: An Overview. *Swarm Intelligence*. 2007;1:33–57.
29. Trelea IC. The particle swarm optimization algorithm: convergence analysis and parameter selection. *Inf Process Lett*. 2003;85:317–25.
30. Dorigo M, Birattari M, Stutzle T. Ant colony optimization. *IEEE Comput Intell Mag*. 2006;1:28–39.
31. Socha K, Dorigo M. Ant colony optimization for continuous domains. *Eur J Oper Res*. 2008;185:1155–73.
32. Goldberg D. Genetic algorithms in search. Optimization and machine learning: Addison-Wesley publishing company. Massachusetts: Reading; 1989.
33. Michalewicz Z. Genetic algorithms + data structures = evolution programs. Berlin: Springer - Verlag; 1996.
34. Grady SA, Hussaini MY, Abdullah MM. Placement of wind turbines using genetic algorithms. *Renew Energy*. 2005;30:259–70.
35. Zhou Y, Tan Y. GPU-based parallel particle swarm optimization. 2009 IEEE Congress on Evolutionary Computation, 2009;1493–1500.
36. Dawson L, Stewart I. Improving Ant Colony Optimization performance on the GPU using CUDA. *IEEE Congress Evolut Comput*. 2013;2013:1901–8. <https://doi.org/10.1109/CEC.2013.6557791>.
37. Barkalov K, Gergel V. Parallel global optimization on GPU. *J Glob Optim*. 2016;66:3–20.
38. Boussaïd I, Lepagnot J, Siarry P. A survey on optimization metaheuristics. *Inf Sci*. 2013;237:82–117.
39. Dokeroglu T, Sevinc E, Kucukyilmaz T, Cosar A. A survey on new generation metaheuristic algorithms. *Comput Ind Eng*. 2019;137: 106040.
40. Hussain K, Salleh MNM, Cheng S, Shi Y. Metaheuristic research: a comprehensive survey. *Artif Intell Rev*. 2019;52:2191–233.
41. de Moura Meneses AA, Marcelo Dornellas, Machado Roberto Schirru, Particle Swarm Optimization applied to the nuclear reload problem of a Pressurized Water Reactor. *Prog Nucl Energy*. 2009;51:319–26.
42. Ranjit S, Shalivahan S. Particle swarm optimization: a new tool to invert geophysical data. *Geophysics* 2007;72.
43. Ourique CO, Biscaia EC, Pinto JC. The use of particle swarm optimization for dynamical analysis in chemical processes. *Comput Chem Eng*. 2002;26:1783–93.
44. Fang H, Zhou J, Wang Z, et al. Hybrid method integrating machine learning and particle swarm optimization for smart chemical process operations. *Front Chem Sci Eng*. 2022;16:274–87.
45. Wachowiak MP, Smolikova R, Zheng Y, Zurada JM, Elmaghraby AS. An approach to multimodal biomedical image registration utilizing particle swarm optimization. *IEEE Trans Evol Comput*. 2004;8:289–301.
46. Marinakis Y, Magdalene Marinaki, Georgios Dounias, Particle swarm optimization for pap-smear diagnosis. *Expert Syst Appl*. 2008;35:1645–56.
47. Park J-B, Jeong Y-W, Shin J-R, Lee KY. An improved particle Swarm optimization for nonconvex economic dispatch problems. *IEEE Trans Power Syst*. 2010;25:156–162166.
48. Liu B, Wang L, Jin YH. An effective PSO-based memetic algorithm for flow shop scheduling. *IEEE Trans Syst Man Cybern Part B (Cybernetics)*. 2007;37:18–27.
49. Yang J, He L, Fu S. An improved PSO-based charging strategy of electric vehicles in electrical distribution grid. *Appl Energy*. 2014;128:82–92.
50. Mistry K, Zhang L, Neoh SC, Lim CP, Fielding B. A micro-GA embedded PSO feature selection approach to intelligent facial emotion recognition. *IEEE Trans Cybern*. 2017;47:1496–509.
51. Han S, Shan X, Fu J, Xu W, Mi H. Industrial robot trajectory planning based on improved pso algorithm. *J Phys Conf Ser*. 2021;1820: 012185.
52. Marini F, Walczak B. Particle swarm optimization (PSO). A tutorial, *Chemometrics and Intelligent Laboratory Systems*. 2015;149:153–65.
53. Jain M, Saihpal V, Singh N, Singh SB. An overview of variants and advancements of PSO algorithm. *Appl Sci*. 2022;12:8392.
54. Stacey A, Jancic M, Grundy I. Particle swarm optimization with mutation. In: 2003 Congress on Evolutionary Computation, 2003. CEC '03., 2003. p. 1425–30.
55. Pant M, Thangaraj R, Abraham A. Particle Swarm optimization using adaptive mutation. In: 2008 19th International Workshop on Database and Expert Systems Applications, 2008. p. 519–23.
56. Higashi N, Iba H. Particle swarm optimization with Gaussian mutation. In: Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS'03 (Cat. No.03EX706), 2003. p. 72–9.
57. Engelbrecht A. Particle swarm optimization: velocity initialization. 2012 IEEE Congress on Evolutionary Computation, 2012;1–8.
58. Liu B, Wang L, Jin YH, Tang F, Huang DX. Improved particle swarm optimization combined with chaos. *Chaos Solitons Fractals*. 2005;25:1261–71.
59. Shi XH, Liang YC, Lee HP, Lu C, Wang LM. An improved GA and a novel PSO-GA based hybrid algorithm. *Inf Process Lett*. 2005;93:255–61.
60. Garg H. A hybrid PSO-GA algorithm for constrained optimization problems. *Appl Math Comput*. 2016;274:292–305.
61. Schutte JF, Reinbolt JA, Fregly BJ, Haftka RT, George AD. Parallel global optimization with the particle swarm algorithm. *Int J Numer Method Engng*. 2004;61:2296–315.
62. Koh B-I, George AD, Haftka RT, Fregly BJ. Parallel asynchronous particle swarm optimization. *Int J Numer Method Engng*. 2006;67:578–95.
63. Venter G, Sobieszcanski-Sobieski J. Parallel particle Swarm Optimization Algorithm accelerated by asynchronous evaluations. *J Aerosp Comput Inf Commun*. 2006;3:123–37.
64. Gaing ZL. Particle swarm optimization to solving the economic dispatch considering the generator constraints. *IEEE Trans Power Syst*. 2003;18:1187–95.
65. Yang X, Yuan J, Yuan J, Mao H. A modified particle swarm optimizer with dynamic adaptation. *Appl Math Comput*. 2007;189:1205–13.
66. Jiang Y, Hu T, Huang C, Wu X. An improved particle swarm optimization algorithm. *Appl Math Comput*. 2007;193:231–9.
67. Bogdanova A, Junior JP, Aranha C. Franken-Swarm: grammatical evolution for the automatic generation of Swarm-like Meta-Heuristics. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2019. p. 411–2.
68. O'Neill M, Ryan C. Grammatical evolution. *IEEE Trans Evol Comput*. 2001;5:349–58.
69. Pan X, Xue L, Lu Y, et al. Hybrid particle swarm optimization with simulated annealing. *Multimed Tools Appl*. 2019;78:29921–36.
70. Mughal MA, Ma Q, Xiao C. Photovoltaic cell parameter estimation using hybrid particle swarm optimization and simulated annealing. *Energies* 2017;10.
71. Lin GH, Zhang J, Liu ZH. Hybrid particle swarm optimization with differential evolution for numerical and engineering optimization. *Int J Autom Comput*. 2018;15:103–14.
72. Li S, Tan M, Tsang IW, Kwok JT-Y. A hybrid PSO-BFGS strategy for global optimization of multimodal functions. *IEEE Trans Syst Man Cybern Part B (Cybernetics)*. 2011;41:1003–14.
73. Wu G, Qiu D, Yu Y, Pedrycz W, Ma M, Li H. Superior solution guided particle swarm optimization combined with local search techniques. *Expert Syst Appl*. 2014;41:7536–48.

74. Gao Z, Yu J, Zhao A, Hu Q, Yang S. Optimal chiller loading by improved parallel particle swarm optimization algorithm for reducing energy consumption. *Int J Refrig*. 2022;136:61–70.
75. Olenšek J, Tuma T, Puhan J, Bürmen Á. A new asynchronous parallel global optimization method based on simulated annealing and differential evolution. *Appl Soft Comput*. 2011;11:1481–9.
76. Regis RG, Shoemaker CA. Parallel stochastic global optimization using radial basis functions. *INFORMS J Comput*. 2009;21:411–26.
77. Park J, Sandberg IW. Universal approximation using radial-basis-function networks. *Neural Comput*. 1991;3:246–57.
78. Alba E, Luque G, Nesmachnow S. Parallel metaheuristics: recent advances and new trends. *Int Trans Oper Res*. 2013;20:1–4.
79. Essaid M, Idoumghar L, Lepagnot J, Bréviliers M. GPU parallelization strategies for metaheuristics: a survey. *Int J Parallel Emergent Distrib Syst*. 2019;34:497–522.
80. Koh BI, George AD, Haftka RT, Fregly BJ. Parallel asynchronous particle swarm optimization. *Int J Numer Method Engng*. 2006;67:578–95.
81. Tewolde GS, Hanna DM, Haskell RE. Multi-swarm parallel PSO: hardware implementation. In: 2009 IEEE Swarm Intelligence Symposium, Nashville, TN, USA, 2009, 2009. p. 60–6.
82. Ouyang A, Tang Z, Zhou X, Xu Y, Pan G, Li K. Parallel hybrid PSO with CUDA for ID heat conduction equation. *Comput Fluids*. 2015;110:198–210.
83. de Campos A, Pozo ATR, Duarte EP. Parallel multi-swarm PSO strategies for solving many objective optimization problems. *J Parallel Distrib Comput*. 2019;126:13–33.
84. Lalwani S, Sharma H, Satapathy SC, Deep K, Bansal JC. A survey on parallel particle swarm optimization algorithms. *Arab J Sci Eng*. 2019;44:2899–923.
85. Eberhart RC, Shi YH. Tracking and optimizing dynamic systems with particle swarms. In: Congress on Evolutionary Computation, Korea, 2001.
86. Charillogis V, Tsoulos IG. Toward an ideal particle swarm optimizer for multidimensional functions. *Information*. 2022;13:217.
87. Powell MJD. A Tolerant Algorithm for Linearly Constrained Optimization Calculations. *Math Program*. 1989;45:547–66.
88. Ali MM, Khompatraporn C, Zabinsky ZB. A numerical evaluation of several stochastic algorithms on selected continuous global optimization test problems. *J Global Optim*. 2005;31:635–72.
89. Floudas CA, Pardalos PM, Adjiman C, Esposito W, Gümüs Z, Harding S, Klepeis J, Meyer C, Schweiger C. Handbook of test problems in local and global optimization. Dordrecht: Kluwer Academic Publishers; 1999.
90. Ali MM, Kaelo P. Improved particle swarm algorithms for global optimization. *Appl Math Comput*. 2008;196:578–93.
91. Koyuncu H, Ceylan R. A PSO based approach: scout particle swarm algorithm for continuous global optimization problems. *J Comput Design Eng*. 2019;6:129–42.
92. Siarry P, Berthiau G, Durdin F, Haussy J. *ACM Trans Math Softw*. 1997;23:209–28.
93. Tsoulos IG, Lagaris IE. GenMin: an enhanced genetic algorithm for global optimization. *Comput Phys Commun*. 2008;178:843–51.
94. Gaviano M, Ksasov DE, Lera D, Sergeyev YD. Software for generation of classes of test functions with known local and global minima for global optimization. *ACM Trans Math Softw*. 2003;29:469–80.
95. Lennard-Jones JE. On the determination of molecular fields. *Proc R Soc Lond A*. 1924;106:463–77.
96. Chandra R, Dagum L, Kohr D, Maydan D, McDonald J, Menon R. Parallel programming in open MP. Morgan Kaufmann Publishers Inc.; 2001.
97. Wall M. GALib: A C++ library of genetic algorithm components. Mechanical Engineering Department. Massachusetts Institute of Technology; 1996. p. 54.
98. Kumar L, Pandey M, Ahirwal MK. Parallel Global Best-Worst Particle Swarm Optimization Algorithm for solving optimization problems. *Appl Soft Comput*. 2023;142: 110329.
99. Bishop C. Neural networks for pattern recognition. Oxford University Press; 1995.
100. Cybenko G. Approximation by superpositions of a sigmoidal function. *Math Control Signals Syst*. 1989;2:303–14.
101. Raymer M, Doom TE, Kuhn LA, Punch WF. Knowledge discovery in medical and biological datasets using a hybrid Bayes classifier/evolutionary algorithm. *IEEE Trans Syst Man Cybernet Part B Cybern*. 2003;33:802–13.
102. Zhong P, Fukushima M. Regularized nonsmooth Newton method for multi-class support vector machines. *Optimiz Methods Softw*. 2007;22:225–36.
103. Little MA, McSharry PE, Hunter EJ, Spielman J, Ramig LO. Suitability of dysphonia measurements for telemonitoring of Parkinson's disease. *IEEE Trans Biomed Eng*. 2009;56:1015–22.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.